THỰC HÀNH BUỔI 3

		Depth-Fist Search			Breadth-First Search			Uniform-Cost Search	
Mê cung	Số nút đã duyệt	Đường đi (giải pháp)	Có phải là giải pháp tối ưu không?	Số nút đã duyệt	Đường đi (giải pháp)	Có phải là giải pháp tối ưu không?	Số nút đã duyệt	Đường đi (giải pháp)	Có phải là giải pháp tối ưu không?
Nhỏ (tiny)	15	10 bước	Không tối ưu	15	8 bước	Tối ưu	15	8 bước	Tối ưu
Trung bình (medium)	146	130 bước	Không tối ưu	269	68 bước	Tối ưu	269	68 bước	Tối ưu
Lớn (big)	390	210 bước	Không tối ưu	620	210 bước	Tối ưu	620	210 bước	Tối ưu

```
# search.py
# ---------
# Licensing Information:  You are free to use or extend these projects for
# educational purposes provided that (1) you do not distribute or publish
# solutions, (2) you retain this notice, and (3) you provide clear
# attribution to UC Berkeley, including a link to http://ai.berkeley.edu.
#
# Attribution Information: The Pacman AI projects were developed at UC Berkeley.
# The core projects and autograders were primarily created by John DeNero
# (denero@cs.berkeley.edu) and Dan Klein (klein@cs.berkeley.edu).
# Student side autograding was added by Brad Miller, Nick Hay, and
# Pieter Abbeel (pabbeel@cs.berkeley.edu).


"""
In search.py, you will implement generic search algorithms which are called by
Pacman agents (in searchAgents.py).
"""
```

```python
import util

class SearchProblem:
    """
    This class outlines the structure of a search problem, but doesn't implement
    any of the methods (in object-oriented terminology: an abstract class).

    You do not need to change anything in this class, ever.
    """

    def getStartState(self):
        """
        Returns the start state for the search problem.
        """
        util.raiseNotDefined()

    def isGoalState(self, state):
        """
          state: Search state

        Returns True if and only if the state is a valid goal state.
        """
        util.raiseNotDefined()

    def getSuccessors(self, state):
        """
          state: Search state

        For a given state, this should return a list of triples, (successor,
        action, stepCost), where 'successor' is a successor to the current
        state, 'action' is the action required to get there, and 'stepCost' is
        the incremental cost of expanding to that successor.
        """
        util.raiseNotDefined()

    def getCostOfActions(self, actions):
        """
         actions: A list of actions to take

        This method returns the total cost of a particular sequence of actions.
        The sequence must be composed of legal moves.
        """
        util.raiseNotDefined()
```

```
def tinyMazeSearch(problem):
    """
    Returns a sequence of moves that solves tinyMaze.  For any other maze, the
    sequence of moves will be incorrect, so only use this for tinyMaze.
    """
    from game import Directions
    s = Directions.SOUTH
    w = Directions.WEST
    return  [s, s, w, s, w, w, s, w]

def depthFirstSearch(problem):
    from util import Stack

    stack = Stack()
    stack.push((problem.getStartState(), []))
    visited = set()

    while not stack.isEmpty():
        state, path = stack.pop()

        if state in visited:
            continue

        visited.add(state)

        if problem.isGoalState(state):
            return path

        for successor, action, cost in problem.getSuccessors(state):
            stack.push((successor, path + [action]))

    return []

def breadthFirstSearch(problem):
    """Search the shallowest nodes in the search tree first."""
    from util import Queue

    queue = Queue()
    queue.push((problem.getStartState(), []))
    visited = set()

    while not queue.isEmpty():
        state, path = queue.pop()

        if state in visited:
```

```
            continue
        visited.add(state)

        if problem.isGoalState(state):
            return path

        for successor, action, cost in problem.getSuccessors(state):
            queue.push((successor, path + [action]))

    return []

def uniformCostSearch(problem):
    """Search the node of least total cost first."""
    from util import PriorityQueue

    pq = PriorityQueue()
    pq.push((problem.getStartState(), [], 0), 0)
    visited = {}

    while not pq.isEmpty():
        state, path, cost = pq.pop()

        if state in visited and visited[state] <= cost:
            continue
        visited[state] = cost

        if problem.isGoalState(state):
            return path

        for successor, action, stepCost in problem.getSuccessors(state):
            newCost = cost + stepCost
            pq.push((successor, path + [action], newCost), newCost)

    return []

def nullHeuristic(state, problem=None):
    """
    A heuristic function estimates the cost from the current state to the nearest
    goal in the provided SearchProblem.  This heuristic is trivial.
    """
    return 0

def aStarSearch(problem, heuristic=nullHeuristic):
    """Search the node that has the lowest combined cost and heuristic first."""
    from util import PriorityQueue
```

```
    pq = PriorityQueue()
    pq.push((problem.getStartState(), [], 0), 0)
    visited = {}

    while not pq.isEmpty():
        state, path, cost = pq.pop()

        if state in visited and visited[state] <= cost:
            continue
        visited[state] = cost

        if problem.isGoalState(state):
            return path

        for successor, action, stepCost in problem.getSuccessors(state):
            newCost = cost + stepCost
            priority = newCost + heuristic(successor, problem)
            pq.push((successor, path + [action], newCost), priority)

    return []

# Abbreviations
bfs = breadthFirstSearch
dfs = depthFirstSearch
astar = aStarSearch
ucs = uniformCostSearch
```